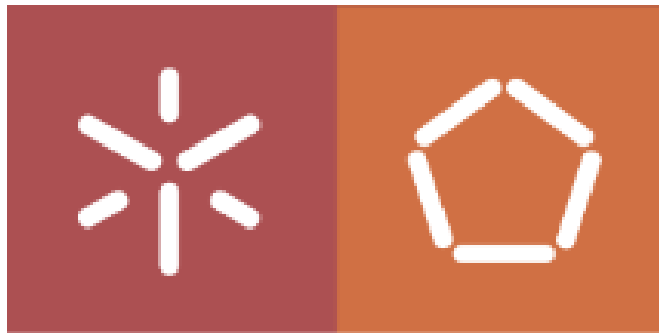


Computação Gráfica

Fase 3 - Transformações Geométricas

Abril 2026



Universidade do Minho
Escola de Engenharia

Grupo: 8

Realizado por: Juliana Sofia Vaz da Silva, a105572

Sofia Beatriz Miranda Couto, a106925

Soraia Filipa Ribeiro Pereira, a106806

Índice

Introdução	3
1. Implementação de VBOs e IBOs	4
1.1. Nova configuração de ficheiros .3d	4
1.1.1. Estrutura IndexedVertices	5
1.2. Classe Model	5
1.3. Implementação	6
1.3.1. Parser	6
1.3.2. Transferência para a GPU	6
1.3.3. Fase de Renderização	7
2. Implementação de patches de Bézier	7
3. Transformações dependentes do tempo	8
3.1. Translação	8
3.2. Rotação	10
3.3. Implementação	10
4. Sistema Solar	11
4.1. Implementação das órbitas e rotações dos planetas e luas	11
4.2. Implementação do cometa	12
4.2.1. Geometria da Órbita	12
5. Testes	13
Conclusão	14

Introdução

Este relatório descreve o desenvolvimento da terceira fase do projeto da unidade curricular de Computação Gráfica, inserida no 2º semestre do 3º ano da Licenciatura em Engenharia Informática, no ano letivo 2025/2026.

Após a introdução de transformações geométricas e gestão de cenas hierárquicas na fase anterior, esta etapa tem como objetivo implementar *VBOs* (*Vertex Buffer Objects*), *patches* de Bézier e transformações dependentes do tempo.

1. Implementação de VBOs e IBOs

Um VBO (*Vertex Buffer Object*) é um buffer de memória de vídeo que armazena informações sobre vértices diretamente na placa gráfica. Nas fases anteriores, era usado o *Immediate Mode*, onde os dados dos vértices eram enviados da CPU para a GPU a cada frame. Com os VBOs, os dados são enviados uma única vez e a GPU mantém esses dados na sua própria memória.

Complementarmente, utiliza-se o IBO (*Index Buffer Object*), um buffer que armazena os índices que definem a ordem de conexão desses vértices. Enquanto o VBO guarda as coordenadas dos pontos, o IBO indica à GPU como montar os triângulos através dos índices dos vértices, permitindo a reutilização de pontos partilhados entre faces. Comparativamente às fases anteriores do projeto, esta combinação é uma otimização de processamento, uma vez que a GPU passa a desenhar a malha de forma indexada e significativamente mais rápida.

Para viabilizar esta transição, optou-se por atualizar os geradores de primitivas para que o cálculo dos índices fosse feito no momento da criação da forma geométrica, sendo depois exportados diretamente para os ficheiros .3d. Esta decisão evitou o custo computacional de calcular índices durante o *parsing* da cena.

Uma vez definida a nova estrutura de armazenamento nos ficheiros .3d (secção 1.1), tornou-se necessária uma entidade capaz de gerir estes dados e comunicar com a placa gráfica. Para esse fim, foi criada a classe `Model` (secção 1.2).

1.1. Nova configuração de ficheiros .3d

Em fases anteriores, nos ficheiros .3d cada triângulo era definido pelos seus três vértices de forma isolada. Isto causava uma enorme repetição de dados, uma vez que os vértices partilhados entre vários triângulos tinham de ser escritos e processados pela GPU múltiplas vezes.

Nesta fase, o formato foi otimizado para uma estrutura indexada. Agora, o ficheiro contém o número de vértices e as coordenadas dos vértices, sem repetições, assim como o número de índices e a ordem dos índices para formar os triângulos.

Na figura 1 encontra-se uma representação visual e explicação das modificações feitas.

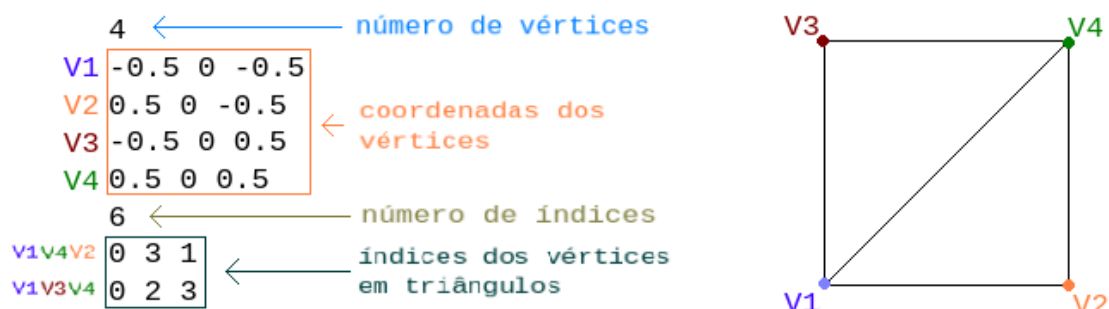


Figura 1 - Explicação da nova configuração e representação visual do plano.

Na transição para esta nova configuração, criou-se uma estrutura de dados para auxiliar na geração das primitivas, explicada abaixo.

1.1.1. Estrutura IndexedVertices

Para suportar a nova configuração indexada, foi implementada a estrutura `IndexedVertices`. Esta estrutura atua como um contentor intermediário, organizando os dados gerados pelas primitivas gráficas no *generator* antes da sua exportação para o ficheiro `.3d`

A estrutura `IndexedVertices` encapsula os seguintes elementos:

- **vertices**: vetor de objetos tipo `Point` com os vértices calculados únicos;
- **indices**: vetor de inteiros que armazena a sequência de índices necessária para definir os triângulos do modelo.

1.2. Classe Model

A classe `Model` funciona como a estrutura de gestão que encapsula a geometria e o estado de renderização de um modelo no *engine*. A estrutura encapsula os seguintes elementos:

- **modelName**: nome do ficheiro `.3d`;
- **vertices**: vetor de objetos tipo `Point` com os vértices lidos no ficheiro `modelName`;
- **indices**: vetor de inteiros com os índices lidos no ficheiro `modelName`;
- **vboID**: identificador único para o buffer que armazena os atributos dos vértices;
- **iboID**: identificador único para o buffer que armazena os índices na ordem em que os vértices devem ser conectados para formar triângulos;
- **numVertices**: tamanho do vetor `vertices`;
- **numIndices**: tamanho do vetor `indices`.

Exemplo explicativo:

Para o excerto de um ficheiro de input XML e o respetivo `.3d` a seguir,

input.xml	plane.3d
<pre><group> <models> <model file="plane.3d" /> </models> </group></pre>	<pre>4 -0.5 0 -0.5 0.5 0 -0.5 -0.5 0 0.5 0.5 0 0.5 6 0 3 1 0 2 3</pre>

é criado um objeto `Model` com os atributos seguintes durante o *parsing*:

<code>modelName</code>	<code>plane.3d</code>
<code>vertices</code>	<code>[Point(-0.5, 0, -0.5), Point(-0.5, 0, -0.5), Point(-0.5, 0, 0.5), Point(0.5, 0, 0.5)]</code>
<code>indices</code>	<code>[0, 3, 1, 0, 2, 3]</code>
<code>vboID</code> ¹	<code>0</code>

¹ São inicializados a zero. O estado muda após a chamada ao método `setupModel()`.

<code>iboID¹</code>	0
<code>numVertices</code>	4
<code>numIndices</code>	6

1.3. Implementação

Para otimizar o desempenho do motor, foi implementado um pipeline que remove a carga de processamento da CPU durante a renderização. Este fluxo garante que a geometria seja processada e armazenada na memória da placa gráfica de forma estruturada, permitindo que o ciclo principal do programa se foque apenas no controlo das transformações e no envio de comandos de desenho. Este pipeline divide-se em três etapas fundamentais: a interpretação dos dados (`Parser`), a transferência para o hardware (GPU) e a exibição final (Renderização).

1.3.1. Parser

Anteriormente, o `Parser` apenas lia um número fixo de vértices e parava. Agora, a função `readModelFile` foi atualizada para um processo de duas etapas:

- **Fase 1 (Vértices):** O parser lê o primeiro número do ficheiro (o `numVertices`), reserva memória através de `vertices.reserve(numVertices)` no vetor e faz um ciclo para ler as coordenadas x, y, z de cada ponto único.
- **Fase 2 (Índices):** Após ler todos os vértices, o `Parser` procura um segundo número (o `numIndices`). Ele faz então a reserva de memória através de `indices.reserve(numIndices)` e um novo ciclo para ler os inteiros que definem a ordem de desenho.

Após a leitura do ficheiro `.3d` é criado um objeto `Model` com o os atributos lidos.

1.3.2. Transferência para a GPU

Uma vez que o `Parser` termina a leitura dos ficheiros e os dados se encontram em estruturas de memória, é necessário transferir os dados para a GPU. Esta transferência é gerida pelo método `setupModel()` da classe `Model` que segue seguinte fluxo:

1. **Geração e Identificação:** O comando `glGenBuffers` solicita à GPU um identificador único. Este ID é armazenado em `vboID` e `iboID` e será a referência futura para aceder a esse espaço de memória específico.
2. **Ligação:** Através de `glBindBuffer`, informa-se o OpenGL que as operações seguintes devem ser aplicadas ao buffer identificado pelo ID gerado. É importante referir que se usou a flag `GL_STATIC_DRAW`, permitindo que a GPU os armazene numa zona de memória de alta performance, otimizando o acesso durante o desenho.
3. **Transferência de Dados:** O comando `glBufferData` efetua a cópia real dos dados.

Este upload ocorre apenas uma vez, durante a fase de inicialização da cena.

1.3.3. Fase de Renderização

A fase de renderização ocorre dentro do ciclo principal do *engine* e é responsável por desenhar a cena no ecrã múltiplas vezes por segundo. Com a implementação de VBOs e IBOs, o fluxo de dados durante cada frame é extremamente reduzido, limitando-se a comandos de ativação e matrizes de transformação. Este processo segue o seguinte fluxo:

1. **Ligação:** No método `drawModel()`, o *engine* utiliza os identificadores guardados para indicar à GPU quais os *buffers* que devem ser utilizados para a renderização imediata.
2. **Transformações Geométricas:** Antes de cada desenho, a classe `Scene` percorre recursivamente o grafo de cena. Para cada grupo, as transformações são aplicadas à matriz atual do OpenGL, garantindo que os modelos são desenhados na posição e escala corretas em relação ao seu nó pai.
3. **Desenho Indexado:** A renderização é executada pelo comando `glDrawElements`. Este comando instrui a GPU a consultar o IBO para saber a ordem exata de conexão dos vértices.

2. Implementação de *patches* de *Bézier*

Nesta fase foi implementada a capacidade de gerar superfícies a partir de *patches* de *Bézier*, permitindo a criação de modelos complexos, os quais são posteriormente convertidos num ficheiro `.3d` composto por triângulos.

É importante entender que um ficheiro `.patch` é composto pelo número de *patches*, 16 índices dos pontos de controlo por *patch* e ainda a lista global dos pontos de controlo, assim como demonstra a figura 2.

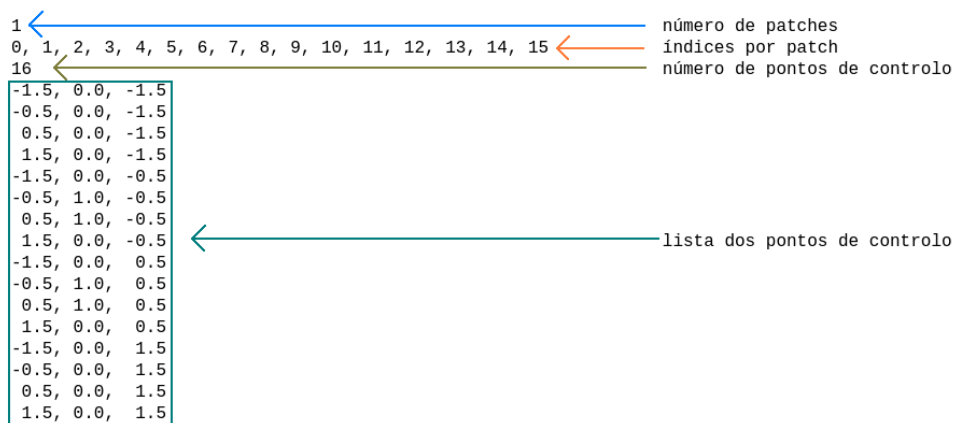


Figura 2 - Estrutura de um ficheiro `.patch`.

Começamos o processo pela leitura e armazenamento destes dados em estruturas `std::vector` e de seguida, iteramos sobre cada *patch* para calcular as coordenadas da superfície `.3d`, tendo em conta um nível de tesselação que é fornecido pelo utilizador.

Para calcular os pontos da superfície, utilizamos a Interpolação de *Bézier* baseada nos polinómios de *Bernstein*. Matematicamente, a equação deste cálculo é:

$$P(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_i^3(u) \cdot B_j^3(v) \cdot P_{ij}$$

Onde:

- $u, v \in [0, 1]$ representam os parâmetros que variam de 0 a 1
- P_{ij} são os pontos de controlo lidos do ficheiro .patch
- $B_i^3(t)$ são os polinómios de *Bernstein* de grau 3

Para gerar a triangulação da superfície de forma otimizada e evitar a duplicação de vértices optámos por utilizar **IndexedVertices** permitindo a reutilização de vértices através de índices, o que melhora a eficiência na renderização, como foi visto na primeira parte do projeto.

Para iniciar este processo, começamos por calcular o espaçamento **delta_tess** e gerar uma grelha 2D apenas com os vértices únicos da superfície.

Feito isto, a função **append_patch_indices** percorre esta grelha 2D e define a ordem correta dos índices (triângulo a triângulo) para ligar os vértices, respeitando a orientação das faces. Estes índices e vértices são depois guardados no ficheiro .3d final.

3. Transformações dependentes do tempo

De forma a permitir a animação de objetos na cena, foram ainda implementadas nesta fase transformações dependentes do tempo. Estas transformações correspondem a rotações e translações ao longo de curvas que ocorrem ciclicamente ao longo de um período de tempo definido no ficheiro XML.

3.1. Translação

Para a implementação das translações dependentes do tempo foram utilizadas curvas cúbicas de *Catmull-Rom*, realizadas através da interpolação de um conjunto de pontos de controlo. Estes pontos de controlo são definidos no ficheiro XML, tal como o tempo necessário para percorrer a curva completa e um campo *align*, indicando se o objeto deve ser alinhado com a curva.

Como é possível observar na figura 1, cada segmento nas curvas cúbicas de *Catmull-Rom* é definido por quatro pontos de controlo, onde os dois pontos centrais P_1 e P_2 são os pontos de controlo que delimitam o segmento.

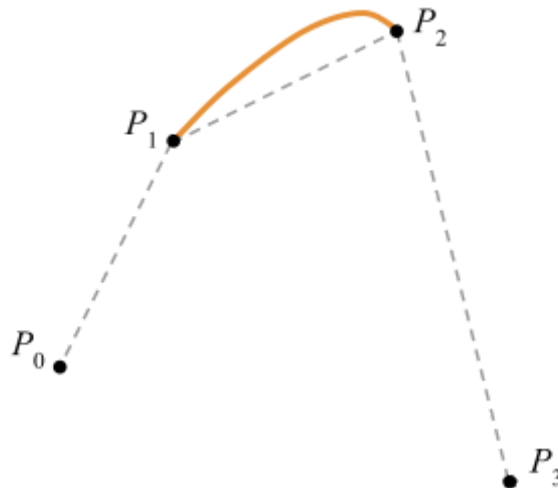


Figura 3: Segmento obtido através de quatro pontos de controlo.

Sendo $t \in [0, 1]$ o parâmetro de interpolação local ao longo do segmento, tem-se que a posição do objeto no segmento no instante t é dada por:

$$P(t) = \begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix} \begin{bmatrix} -0.5 & 1.5 & -1.5 & 0.5 \\ 1 & -2.5 & 2 & -0.5 \\ -0.5 & 0 & 0.5 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix}$$

onde a matriz 4x4 representada corresponde à matriz de *Catmull-Rom*. Repetindo este cálculo para cada componente do ponto, x , y e z , obtém-se o vetor utilizado para construir a matriz de translação a ser aplicada ao objeto.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ P_x & P_y & P_z & 1 \end{bmatrix}$$

Nos casos em que o objeto tem de estar alinhado com a curva, a sua rotação é determinada pela tangente da curva, dada pela derivada da sua posição em t .

$$P'(t) = \begin{bmatrix} 3t^2 & 2t & 1 & 0 \end{bmatrix} \begin{bmatrix} -0.5 & 1.5 & -1.5 & 0.5 \\ 1 & -2.5 & 2 & -0.5 \\ -0.5 & 0 & 0.5 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix}$$

De forma semelhante à posição, este cálculo é feito para todas as componentes do ponto, obtendo-se o vetor tangente à curva. Sendo X_i o vetor tangente à curva, Z_i o vetor lateral, obtido pelo produto externo da tangente atual e do vetor up do instante anterior, e Y_i o novo vetor up (perpendicular aos dois anteriores), isto é,

$$\begin{aligned} \vec{X}_i &= P'(t) \\ \vec{Z}_i &= X_i \times \vec{Y}_{i-1} \\ \vec{Y}_i &= \vec{Z}_i \times \vec{X}_i \text{ com } \vec{Y}_0 = (0, 1, 0) \end{aligned}$$

é possível construir a partir destes vetores a matriz de rotação do objeto:

$$\begin{bmatrix} X_x & Y_x & Z_x & 0 \\ X_y & Y_y & Z_y & 0 \\ X_z & Y_z & Z_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

No programa, ambas a posição e a derivada num instante normalizado t são calculadas pela função `getCatmullRomPoint` da classe `CatmullRomCurve`. Esta classe também possui a função `getGlobalCatmullRomPoint` que recebe um parâmetro global gt e determina o índice do segmento em que se encontra i da seguinte forma:

$$i = \lfloor gt \times N_{\text{pontos}} \rfloor$$

A partir do índice do segmento a função calcula também o parâmetro de interpolação local $t \in [0, 1]$:

$$t = (gt \times N_{\text{pontos}}) - i$$

Os índices dos pontos de controlo que são considerados para um segmento de índice i são obtidos calculando o resto da divisão inteira pelo número total de pontos N_{pontos} de forma a garantir que a curva seja fechada:

- $P_0 = (i + N_{\text{pontos}} - 1) \bmod N_{\text{pontos}}$
- $P_1 = i \bmod N_{\text{pontos}}$
- $P_2 = (i + 1) \bmod N_{\text{pontos}}$
- $P_3 = (i + 2) \bmod N_{\text{pontos}}$

3.2. Rotação

Já a implementação da rotação dependente do tempo é muito mais simples, sendo apenas necessário calcular o seu ângulo com base no instante atual. Sendo t_r o tempo necessário para completar uma rotação, tem-se que o ângulo de rotação num dado instante t é dado por:

$$\theta(t) = \frac{t \times 2\pi}{t_r}$$

O cálculo da matriz de rotação é feito de maneira igual ao da rotação estática.

3.3. Implementação

Como nem todas as transformações são estáticas, não é mais viável o uso de uma matriz de transformação final para a sua representação na classe `Group`, uma vez que é necessário calculá-la a cada frame para as transformações dependentes do tempo. Passou a ser então preciso guardar a informação necessária para este cálculo. Para tal, optou-se por criar uma classe abstrata `Transformation`, com os métodos abstratos `getMatrix(time)` e `isStatic`, e a classe `Group` passou a conter um vetor de ponteiros para objetos do tipo `Transformation`. Esta classe é herdada pelas classes `StaticTransformation`, `CurveTransformation` e `RotationTimeTransformation`, que implementam os métodos de acordo com o tipo de transformação.

As classes `CurveTransformation` e `RotationTimeTransformation` correspondem à translação ao longo da curva e à rotação dependente do tempo, respetivamente. A classe `CurveTransformation` guarda uma instância da classe utilitária `CatmullRomCurve`, mencionada anteriormente. Para obter a posição e a derivada da curva necessários para o cálculo da matriz de transformação num dado instante, é chamado o método `getGlobalCatmullRomPoint`. Esta classe também inclui métodos para inicializar um VBO com os pontos da curva e para desenhar a curva usando esse VBO.

Já a classe `StaticTransformation` representa as transformações estáticas (translação,

rotação, escala), podendo corresponder à acumulação de várias transformações estáticas, como implementado na fase anterior, caso a ordem em que aparecem no grupo seja sucessiva. Nesse caso a matriz de transformação é calculada durante o parsing do ficheiro XML, multiplicando sucessivamente as matrizes de cada transformação estática. Caso contrário, ou seja, caso haja, por exemplo, uma transformação dependente do tempo entre duas transformações estáticas, estas são guardadas como transformações estáticas individuais, sem acumulação, para assegurar a ordem correta das transformações.

4. Sistema Solar

Para validar e demonstrar as funcionalidades implementadas nesta fase, foi desenvolvida uma simulação do sistema solar. Este cenário permite testar a utilização de VBOs e das novas animações através da implementação de um cometa e das rotações e translações dos planetas e luas em torno do sol e sobre si mesmos. Implementamos ainda a opção de renderizar ou não os eixos na cena de acordo com o que o xml indicar, através de um campo “*axis*” booleano que, por *default*, é falso.

```
<axis show="true"/>
```

4.1. Implementação das órbitas e rotações dos planetas e luas

Para conseguir implementar a órbita dos planetas e das luas, bem como a rotação dos mesmos sobre o seu próprio eixo, acrescentou-se aos dicionários definidos na fase anterior os tempos de rotação e de órbita em horas. À exceção de algumas das luas de Júpiter e Saturno, cujos tempos foram gerados aleatoriamente, os tempos utilizados são proporcionais aos tempos reais.

Com o intuito de facilitar a visualização destas rotações, usou-se uma proporção de 1 hora real para 1 segundo no XML, ou seja, um planeta que leva 24 horas para completar uma rotação sobre o seu próprio eixo tem um tempo de rotação de 24 segundos. Já no caso do tempo de órbita, para atenuar a disparidade entre os tempos de órbita e os tempos de rotação sobre o próprio eixo, usou-se uma proporção de 1 hora real para 1/500 segundos no XML. Ainda assim, acrescentou-se um parâmetro `time_scale` ao *script* para permitir personalizar estas escalas com maior detalhe. A ordem dos argumentos opcionais recebidos pelo *script* passou então a ser:

```
./generate_solar_system.sh <sun_scale> <planet_scale> <dist_scale>  
<angle_variation> <time_scale>
```

Na incorporação das transformações de rotação dependentes de tempo nas transformações já existentes para os grupos dos planetas e das luas, foi importante ter em conta a ordem com que estas são aplicadas em cada grupo, para garantir que estes movimentos são corretamente representados. Para os planetas, a ordem de transformações é:

1. Rotação estática para definir a fase inicial da órbita
2. Rotação dependente do tempo para definir a órbita
3. Translação para posicionar o planeta à distância correta do Sol
4. Rotação dependente do tempo para definir a rotação sobre o próprio eixo
5. Escala para ajustar o tamanho do planeta

Para as luas, a ordem de transformações é:

1. Rotação estática para definir a fase inicial da órbita em torno do planeta
2. Rotação dependente do tempo para definir a órbita em torno do planeta

3. Translação para posicionar a lua à distância correta do planeta
4. Rotação dependente do tempo para definir a rotação sobre o próprio eixo
5. Escala para ajustar o tamanho da lua

4.2. Implementação do cometa

Para cumprir os requisitos desta etapa, foi adicionado um cometa à simulação do sistema solar. A sua implementação destaca-se pela utilização de interpolação por curvas de *Catmull-Rom*, permitindo que o objeto percorra uma trajetória não linear definida por pontos no espaço.

A implementação do cometa diverge da dos planetas (que utilizam órbitas circulares simples) para simular o comportamento de um corpo celeste com uma órbita elíptica.

4.2.1. Geometria da Órbita

Diferente dos planetas, que mantêm uma órbita circular, o cometa tem uma órbita elíptica alternando a distância ao centro do sol entre o afélio (ponto mais distante) e o periélio (ponto mais próximo).

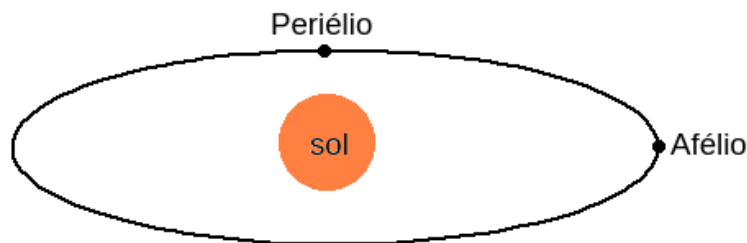


Figura 4: Apresentação visual dos pontos periélio e afélio.

Optou-se por uma elipse centrada no sol para a trajetória do cometa. Esta escolha permitiu focar o desenvolvimento na correta geração de pontos e na fluidez do movimento através da curva.

No código, estes dois pontos são calculados através das suas distâncias, sendo:

$$\begin{aligned} \text{afelio_dist} &= \text{escala_sol} * 0.1 * 1.2 \\ \text{perielio_dist} &= \text{escala_sol} * 0.1 * 2 \end{aligned}$$

Para que a função `getGlobalCatmullRomPoint` consiga processar a trajetória, o script gera uma lista de 14 pontos. A escolha deste número prende-se com a necessidade de garantir resolução geométrica suficiente para que a interpolação cúbica resulte numa elipse visualmente suave.

Para gerar cada um destes pontos de controlo ao redor da elipse, são utilizadas as seguintes fórmulas:

$$\begin{aligned} x &= \text{afelio_dist} * \cos(t) \\ y &= 0.3 * \sin(t * 0.5) \\ z &= \text{perielio_dist} * \sin(t) \end{aligned}$$

Onde:

- $t = \frac{i}{13} * 2\pi$;
- t : Representa o parâmetro angular da elipse, variando entre 0 e 2π para completar uma volta total;
- i : É o índice do ponto de controlo (variando de 0 a 13 para gerar os 14 pontos);

A componente “y” introduz uma ligeira oscilação vertical, simulando a inclinação orbital característica destes astros. O algoritmo de *Catmull-Rom* utiliza a lógica de fecho da curva para garantir que a transição entre o último e o primeiro ponto seja perfeitamente fluida, completando o ciclo elíptico.

Por fim, definiu-se um tempo base de 50.0 unidades, ajustado pelo multiplicador `time_scale`, para definir o período orbital completo do cometa.

5. Testes

Para testar as transformações implementadas nesta fase, utilizamos os ficheiros de teste disponibilizados pela equipa docente.

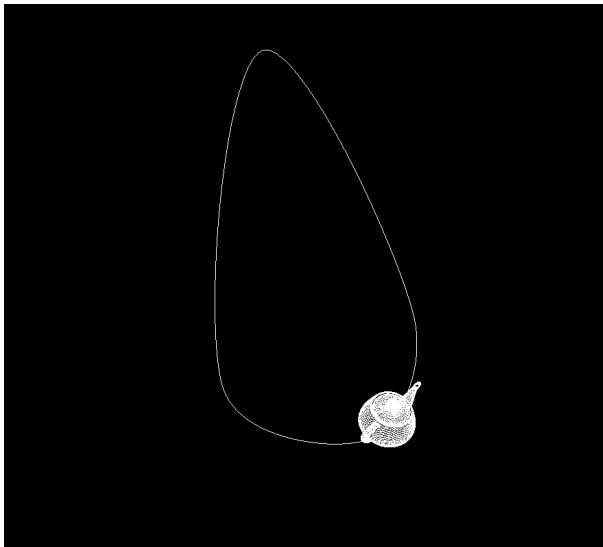


Figura 5 - Teste 1.

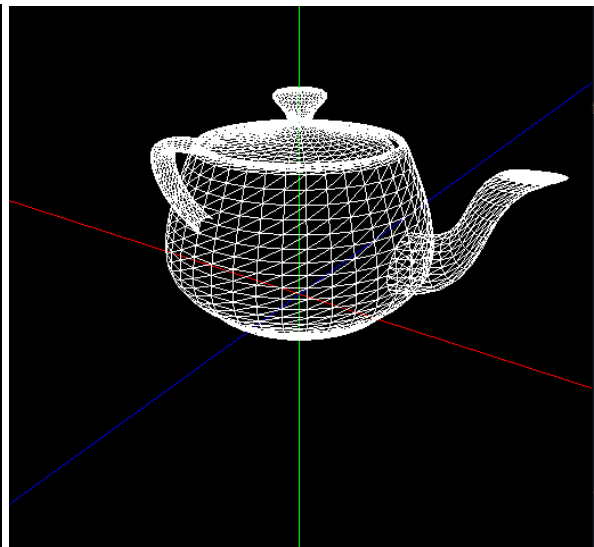


Figura 6 - Teste 2.

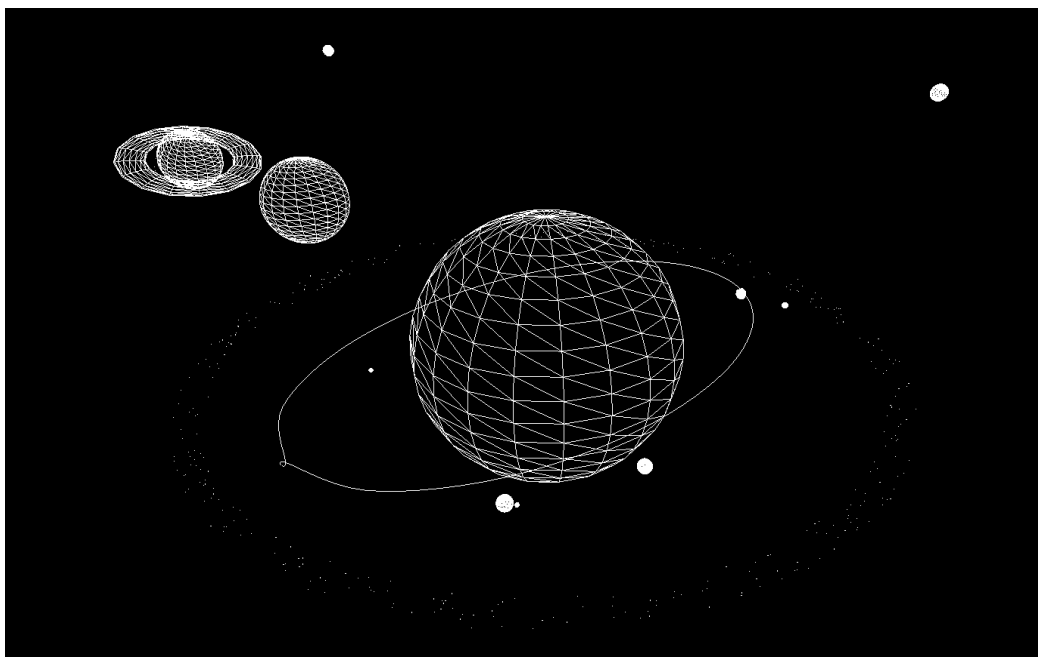


Figura 5 - Sistema solar gerado com `./generate_solar_system.sh 0.9 4.5 0.3 -90`.

Conclusão

Consideramos que a terceira fase do projeto foi implementada com sucesso. A utilização de VBOs e IBOs, juntamente com o suporte para a criação de superfícies mais complexas com recurso a *patches* de *Bézier* permitiram tornar o *engine* mais eficiente e capaz de representar modelos mais detalhados.

Além disso, a introdução de transformações dependentes do tempo tornou a cena dinâmica, permitindo a animação de objetos através de rotações contínuas e translações ao longo de curvas de *Catmull-Rom*. Estas funcionalidades foram aplicadas com sucesso na modelação do sistema solar que passou a incluir órbitas animadas e ainda o movimento de um cometa.

Realizamos ainda vários testes que tiveram os resultados esperados, garantindo assim a capacidade do *engine* de lidar com animações e estruturas mais complexas.

Em suma, os objetivos definidos para a terceira fase do projeto foram concluídos com sucesso e garantimos uma base sólida para a próxima e última etapa do trabalho.